Índice

Objetivos de la sesión
Conocimientos y materiales necesarios
1. El principio de localidad
2. Jerarquía de memoria sin caché
3. Jerarquía de memoria con caché
4. Tiempo de acceso a la jerarquía de memoria
5. Simulación de caché basada en traza
Ejercicios

Introducción a la jerarquía de memoria

Área de Arquitectura y Tecnología de Computadores – Versión 1.2.12009, 25/09/2023

Objetivos de la sesión

El objetivo general de esta sesión es comprender el efecto que tiene la jerarquía de memoria sobre el rendimiento del computador. Los objetivos específicos que se persiguen en esta sesión son:

- Resaltar las mejoras obtenidas en tiempo de acceso a memoria que introduce la memoria caché.
- Ilustrar el principio de localidad, tanto espacial como temporal, que es la base del funcionamiento de la jerarquía de memoria.
- Introducir la simulación de caché basada en traza del simulador *pyMIPS64*.

Conocimientos y materiales necesarios

Para poder realizar esta sesión, el alumno debe:

- Acudir al laboratorio con los apuntes de teoría relativos a la jerarquía de memoria. Es especialmente importante haber leído los apuntes y conocer los conceptos básicos relacionados con la memoria caché.
- Durante la sesión de plantearán una serie de preguntas que puedes responder en el correspondiente cuestionario en el campus virtual. Puedes abrir el cuestionario en otra pestaña del navegador pinchando en el enlace mientras mantienes pulsada la tecla `Ctrl`.

Trabajarás en un entorno Windows, así que no es necesario que arranques tu máquina GNU/Linux.

- Descarga los archivos de la práctica y descomprímelos en tu directorio de trabajo.

1. El principio de localidad

El principio de localidad es la base sobre la que descansa la jerarquía de memoria, pues permite que el tiempo medio de acceso a la jerarquía de memoria sea inferior al de la memoria principal, acercándose incluso al de la memoria caché.

Se ilustrará este principio con el programa ensamblador `3-1upper-case.asm` para el simulador pyMIPS64. Este programa, cuyo listado aparece a continuación, tiene como misión convertir a mayúsculas los caracteres de una cadena. Para realizar su trabajo el programa lee el carácter de la cadena, realiza la operación AND del carácter con la máscara adecuada[1] y, finalmente, guarda el carácter modificado en la cadena. El final de la cadena viene dado por el carácter de código ASCII 0.

```
;
; Converts a null-terminated ASCII string to upper-case
;
        .data
string: byte 'A_simple_string', 0

        .code
main:
        xor   r10, r10, r10    ; index of next character

loop:
        lb    r20, string(r10) ; next character
        beqz  r20, final       ; end if 0 character
        andi  r20, r20, DFh    ; mask to convert to upper-case
        sb    r20, string(r10) ; store the converted character
        daddi r10, r10, 1      ; next character
        j     loop
final:
        nop
```

- Arranca el simulador *pyMIPS64* y carga el archivo fuente `3-1upper-case.asm` de la forma habitual.
- Abre la hoja de cálculo `3-1mem-trace.xlsx`. Según vayas simulando la ejecución del programa, además de responder a las preguntas, debes rellenar las casillas para reflejar los accesos a memoria. Para cada acceso a la memoria de código, en la gráfica de código debes poner un `x` en las coordenadas (número de acceso, dirección de código). Asimismo, en la gráfica de datos debes poner una `x` en las coordenadas (número de acceso, dirección de datos). Debes tener en cuenta que los accesos a datos y código emplean la misma numeración, por lo que un número de acceso será de código o de datos, pero no de los dos al mismo tiempo.
- Simula un ciclo de reloj. La etapa `IF` está activa en el ciclo de reloj 1 durante el cual busca la instrucción `xor r10, r10, r10`, que se encuentra en la dirección 0800h.
- Simula cuatro ciclos de reloj más; hasta el ciclo de reloj 5. En el ciclo de reloj 2 la etapa `IF` está activa buscando ahora la instrucción `lb r20, string(r10)`. ¿A qué dirección de memoria accede? Responde en el cuestionario: pregunta 1.
- En el ciclo 3 la etapa `IF` accede a la memoria para buscar la instrucción `beqz r20, final`. ¿A qué dirección de memoria accede? Responde en el cuestionario: pregunta 2. La etapa `IF` al estar detenida en los ciclos de reloj 3 y 4 no puede escribir en su registro de segmentación de salida (`IF/ID`) y tiene que esperar al ciclo 5 para poder escribirlo. Cuando las etapas `IF` o `MEM` aparecen de forma consecutiva en varios ciclos de reloj para la misma instrucción, sólo se accede a memoria en el primero de los ciclos (en el que no sufra una detención estructural).
- Simula otro ciclo de reloj. En el ciclo de reloj 6 la etapa `IF` accede a la memoria para buscar la instrucción `andi r20, r20, DFh`. ¿A qué rango de direcciones de memoria accede? Responde en el cuestionario: pregunta 3.
- Simula otro ciclo de reloj. En el ciclo de reloj 7 la etapa `MEM` accede a la memoria para leer el primer carácter de la cadena que se encuentra en la dirección `string(r10)`. ¿A qué dirección de memoria accede? Responde en el cuestionario: pregunta 4.
- Simula hasta el ciclo 17. En los ciclos 12, 13, 16 y 17 la etapa `IF` accede a la memoria de código. ¿A qué direcciones de memoria accede? Responde en el cuestionario: pregunta 5.
- En el ciclo 17 la etapa `MEM` accede a la memoria de datos para escribir el primer carácter en mayúsculas. ¿A qué dirección de memoria accede? Responde en el cuestionario: pregunta 6.
- Simula hasta el ciclo de reloj 32. Refleja en las gráficas de código y datos los accesos realizados.
- Una vez realizadas varias iteraciones puedes terminar de completar las gráficas siguiendo el mismo patrón.

A partir de las gráficas con los accesos a memoria se plantean una serie de cuestiones relacionadas con la localidad del programa anterior.

- Teniendo en cuenta la representación de la secuencia de accesos a memoria, ¿crees que ésta es aleatoria, o por el contrario sigue algún tipo de patrón? Responde en el cuestionario: pregunta 7.
- Indica un ejemplo de localidad espacial en el acceso al código del programa anterior. Responde en el cuestionario: pregunta 8.
- Indica un ejemplo de localidad temporal en el acceso al código del programa anterior. Responde en el cuestionario: pregunta 9.
- Indica un ejemplo de localidad espacial en el acceso a los datos del programa anterior. Responde en el cuestionario: pregunta 10.
- Indica un ejemplo de localidad temporal en el acceso a los datos del programa anterior. Responde en el cuestionario: pregunta 11.

2. Jerarquía de memoria sin caché

En este apartado se configurará la memoria del simulador MIPS64 únicamente con memoria principal para obtener el tiempo de ejecución sin caché. El objetivo es comparar posteriormente el tiempo de ejecución del programa con caché.

- Carga de nuevo el archivo fuente `3-1upper-case.asm` para reiniciar la simulación.

Hasta ahora, los archivos cargados se simulaban ignorando los efectos de la memoria, para lo cual se suponía que cada acceso a memoria requería un ciclo de reloj. Además, se suponía una arquitectura Harvard de memoria, es decir, se disponía de una memoria de código y una de datos, lo que impedía que ocurriera detenciones estructurales en los accesos a memoria por parte de las etapas `IF` y `MEM`.

- Pulsa con el ratón sobre el icono en la barra de menú. Aparte de la pestaña de configuración de microarquitectura que hemos usado en las sesiones de CPU, verás una segunda pestaña que permite configurar la memoria.
- Selecciona la pestaña de memoria y observa la configuración por defecto de la misma. Por un lado, la arquitectura de memoria es la habitual de MIPS64: la arquitectura Harvard. Por otro lado, la memoria caché está deshabilitada y el tiempo de acceso a memoria principal es de 1 ciclo de reloj. Un tiempo de acceso a memoria principal de 1 ciclo de reloj no se correponde con la realidad, pues en la práctica cada acceso a memoria principal suele requerir múltiples ciclos de reloj.
- Configura el tiempo de acceso a memoria principal con 8 ciclos de reloj y guarda la configuración pulsando sobre el botón [**OK**].
- Simula la primera instrucción completa pulsando sobre el icono ▶| de la barra de menú.
- Pulsa sobre el icono ⚙ de la barra de menú y activa la opción `Show instruction timeline details`. De esta forma el simulador muestra que la etapa `IF` está detenida durante los 7 primeros ciclos. Estas detenciones son de un nuevo tipo: el tipo `MEM`. Este tipo de detenciones ocurren cuando la CPU está esperando a que la memoria complete una operación de lectura o escritura. En este caso, durante los 7 primeros ciclos está esperando a que la memoria le proporcione el código de la primera instrucción y, finalmente, ya está disponible en el octavo. Esta situación se muestra en la figura 1.

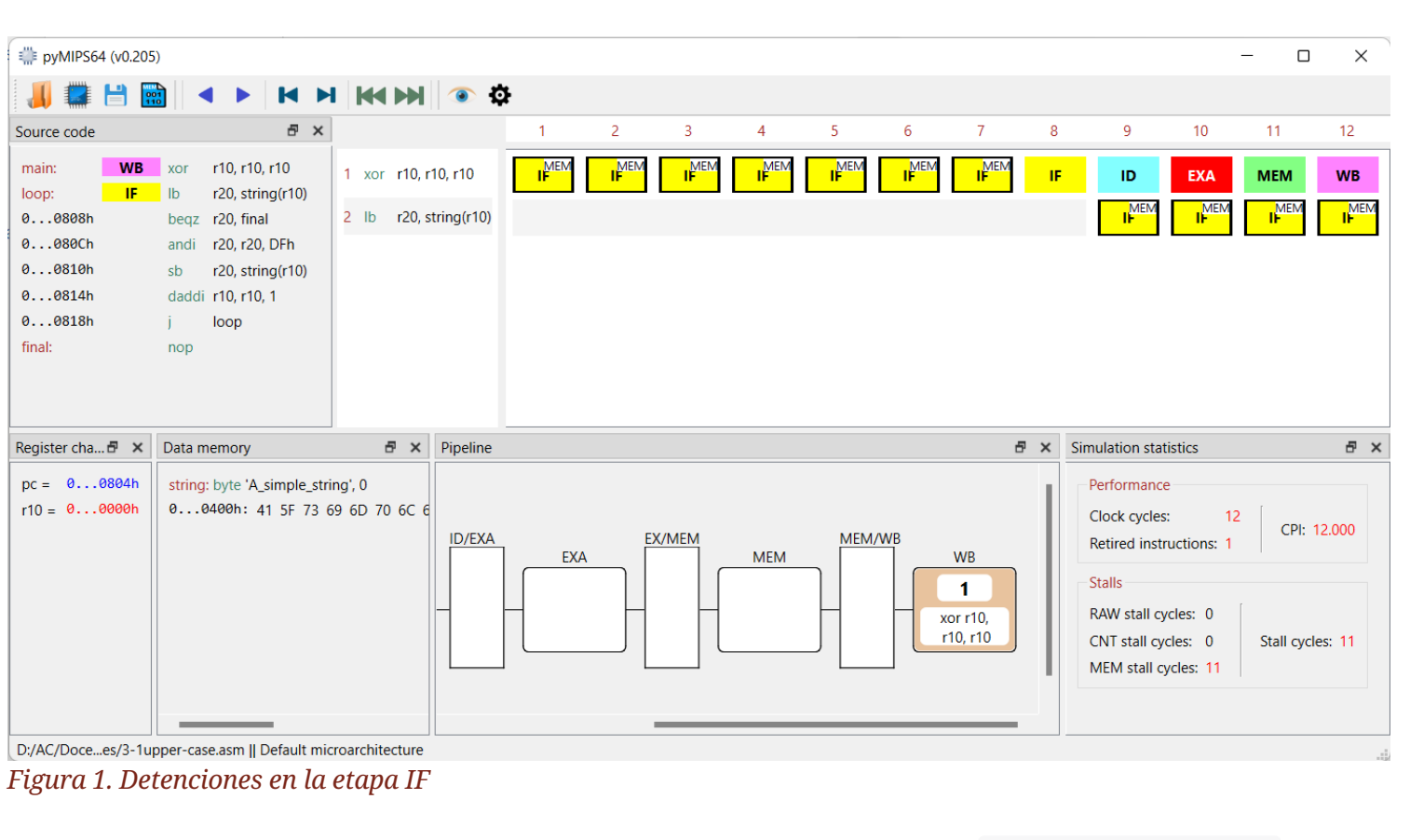

Figura 1. Detenciones en la etapa IF

- Pulsa de nuevo sobre el icono ▶| para simular la segunda instrucción: `lb r20, string(r10)`. En esta instrucción la CPU lee el primer byte de la cadena y lo lleva al registro `r20`.
- Observa que aparecen los mismos 7 ciclos de detención en la etapa `MEM`. En este caso, durante la lectura del primer carácter de la cadena.
- Pulsa ahora sobre el icono ▶▶| para simular el programa completo. Tendrás que esperar unos segundos hasta que termine la simulación. ¿Cuántos ciclos de reloj requiere el programa para su ejecución? Responde en el cuestionario: pregunta 12.

3. Jerarquía de memoria con caché

Para mostrar la influencia de la caché sobre el tiempo de respuesta se plantea una nueva configuración del sistema de memoria. El sistema de memoria será una jerarquía de memoria formada por la caché y la memoria principal.

- En la configuración de la microarquitectura/memoria selecciona la pestaña `Memory` y activa la casilla `Cache enable`. Se muestran los parámetros por defecto de la caché y un mensaje de error indicando que la configuración de memoria es inválida. La razón es que el tiempo necesario para satisfacer un fallo de caché, `Cache miss time (cycles)`, es inferior al tiempo de acceso a la memoria principal, `Main memory access time (cycles)`, y eso no tiene sentido.
- Configura el tiempo de fallo de caché con el mismo valor que el tiempo de acceso a memoria principal.

Una vez que la configuración de caché es correcta se muestra un grupo de información `Cache info` con esta información:

- Tamaño de las cachés de código y datos o unificada.
- Estrategia de correspondencia.
- División en bits de la dirección.
- Tiempo de fallo de caché.
- Tiempo de actualización. Este tiempo siempre coincidirá con el de fallo de caché, pues se supone que se tarda el mismo tiempo en llevar un bloque de memoria caché a memoria principal que llevarlo de memoria principal a caché.

Pulsa el botón [**OK**] para guardar la configuración actual. La simulación se reinicia y aparece un nuevo panel en el cual puede visualizarse el estado actual de la caché.

- Guarda el programa junto con la configuración de la microarquitectura/memoria con el nombre `3-1upper-case-config.asm`. Puedes hacer esta operación pulsando sobre el botón 💾.
- Pulsa 8 veces el botón ▶ para simular 8 ciclos de reloj. Si comparas el estado de la simulación con el que teníamos en los 8 primeros ciclos de la figura 1, parece que nada ha cambiado a pesar de haber introducido la caché. Seguimos teniendo 7 ciclos de detención en la etapa `IF` en los cuales la CPU tiene que esperar por la memoria. ¿Qué crees que ha ocurrido? Responde en el cuestionario: pregunta 13. Si tienes alguna duda simula un ciclo de reloj hacia atrás pulsando el icono ◀ y después un ciclo de reloj hacia adelante pulsando sobre el icono ▶. Durante el proceso observa los cambios en la caché.
- La diferencia la marca la caché cuando se produce un acierto de caché. Simula un nuevo ciclo de reloj. ¿Cuántos ciclos de reloj son necesarios en la etapa `IF` para la búsqueda de la siguiente instrucción, `lb r20, string(r10)`? Responde en el cuestionario: pregunta 14. Compara ese valor con los 8 ciclos de la figura 1, cuando no teníamos caché.
- Simula hasta el final. ¿Cuántos ciclos de reloj requiere ahora el programa para su ejecución? Responde en el cuestionario: pregunta 15. ¿Cuál es la aceleración en la ejecución del programa que proporciona la memoria caché? Responde en el cuestionario: pregunta 16

4. Tiempo de acceso a la jerarquía de memoria

En el apartado anterior se mostraron dos escenarios posibles en el acceso a memoria: fallo de caché y acierto de caché. Los fallos de caché ocurrían sin reemplazo y por lo tanto sin actualización. La razón es la simplicidad del programa en comparación al tamaño de la caché. Para forzar nuevos escenarios vamos a considerar una caché más pequeña y además vamos a usar una arquitectura *von Neumann*, es decir, vamos a tener una caché unificada que almacenará tanto código como datos.

- Cambia la configuración de la memoria para que utilice una arquitectura *von Neumann*.
- Cambia además el número de líneas de caché para que sea 4 en lugar de 32. La simulación se reinicia y el panel `Cache memory` muestra el nuevo estado de la caché.
- Simula las dos primeras instrucciones. El estado del simulador será el mostrado en la figura 2.

Observarás que algunas etapas tienen detenciones marcadas con un texto en gris. Por ejemplo, en el ciclo de reloj 10 la etapa `IF` sufre una detención de tipo `MEM` marcada de esta forma. Esta detención `MEM`, no se contabiliza en el panel de estadísticas, pues en el mismo ciclo se detiene una instrucción anterior, la instrucción `lb r20, string(r10)` en la etapa `ID`. Este es el criterio que sigue el simulador para no contabilizar más de una detención en un ciclo de reloj.

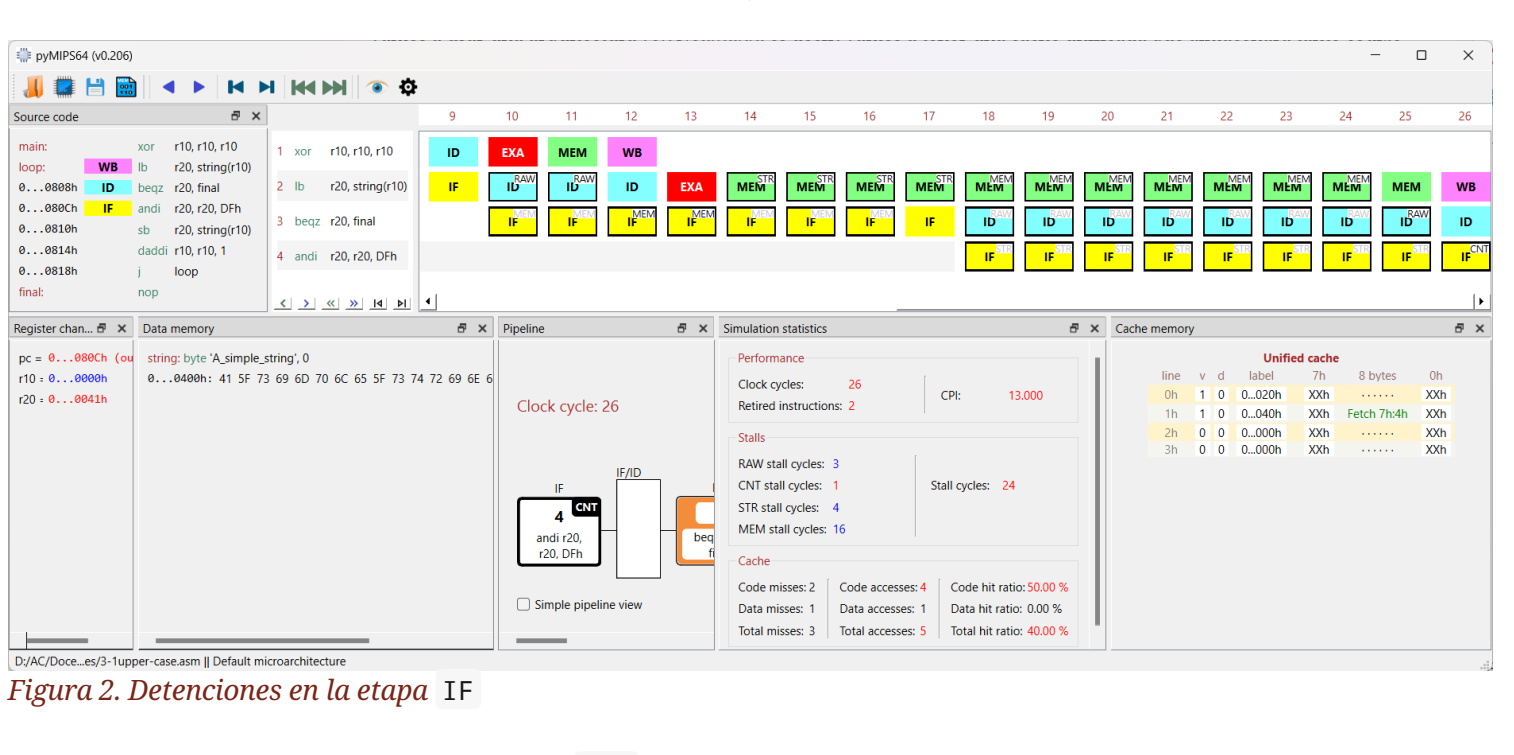

Figura 2. Detenciones en la etapa `IF`

- ¿Qué tipo de detención sufre la etapa `MEM` en los ciclos 14 a 17? Responde en el cuestionario: pregunta 17. Si no encuentras la repuesta puedes activar la opción `Show instruction timeline details`. ¿Por qué se produce esa detención? Responde en el cuestionario: pregunta 18.

Esa detención no es exclusiva de la etapa `MEM`. También la puede sufrir la otra etapa que accede a memoria, la etapa `IF`, como puedes observar en los ciclos 18 a 25.

Hasta ahora en la configuración del simulador se ha empleado como estrategia de escritura *write-back* y *write-allocate* ante fallos de escritura, pero son posibles otras.

- Abre el fichero `3-1upper-case-config.asm` que guardaste previamente y cambia la configuración de la memoria para que utilice *write-through* y *no-write-allocate*. Con esta nueva configuración cuando se produce acierto de caché en escritura se escribe a la vez en la memoria caché y en la memoria principal. Cuando se produce fallo de caché en escritura no se lleva el bloque de memoria principal a caché y sólo se escribe en memoria principal.
- Cambia el tamaño de las líneas de caché para que sean de 16 bytes en lugar de 8 bytes y multiplica por 2 el tiempo de fallo de caché para que sea acorde al nuevo tamaño de línea. Guarda la nueva configuración.
- Con esta nueva configuración, ¿puede haber actualización de la memoria principal durante algún acceso a memoria? ¿Por qué? Responde en el cuestionario: pregunta 19.
- ¿Cuál será el tiempo de acceso a memoria durante un fallo de caché en escritura? Responde en el cuestionario: pregunta 20. El primer fallo de caché en escritura ocurre en la etapa `MEM` cuando se ejecuta por primera vez la instrucción `sb r20, string(r10)`. Ejecuta hasta completar dicha instrucción y comprueba tu respuesta.
- La siguiente ocasión que se ejecuta la instrucción `sb r20, string(r10)` se produce un acierto de caché en escritura. ¿Cuál será el tiempo de acceso a memoria durante esa segunda ejecución de la instrucción `sb r20, string(r10)`? Responde en el cuestionario: pregunta 21. Para comprobar la respuesta continúa la ejecución hasta completar por segunda vez dicha instrucción.

En el acceso a memoria podemos encontrarnos con otros escenarios además de los mostrados dependiendo de la configuración. La figura 3 muestra el tiempo de acceso a memoria en todos los escenarios posibles expresado en ciclos de reloj. Debe observarse que el tiempo de acceso a la caché, 1 ciclo de reloj, se desprecia cuando se suma a otros tiempos mayores, como es el caso del tiempo de fallo de caché, actualización, o escritura en memoria principal. Los números empleados en el simulador son deliberadamente bajos para evitar simulaciones muy largas. Aunque en la práctica no son realistas, se utilizan para facilitar la compresión de las simulaciones.

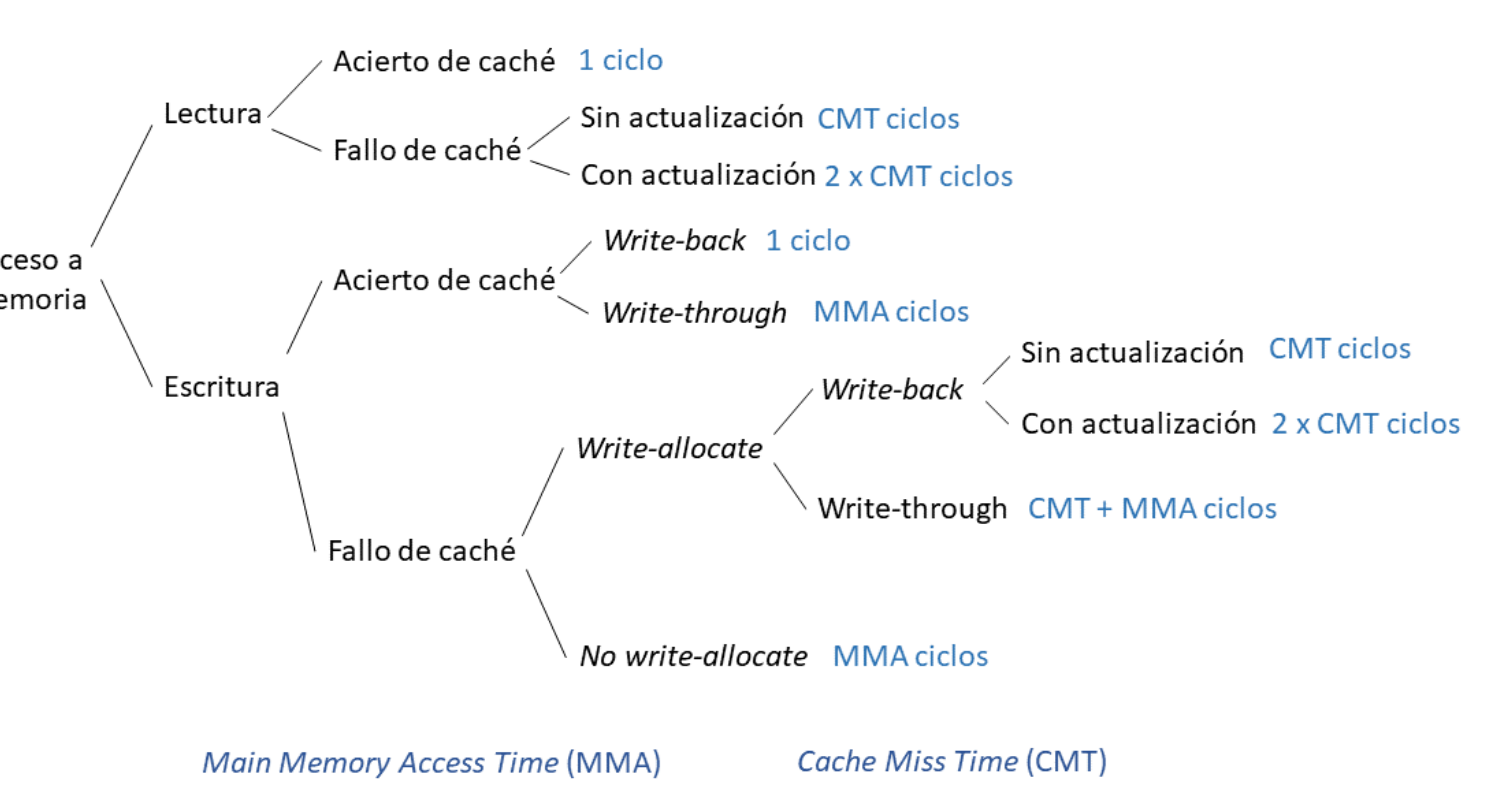

Figura 3. Tiempos de acceso a la jerarquía de memoria

5. Simulación de caché basada en traza

El simulador `pyMIPS64` dispone además de un modo de simulación de caché basado en traza. En este modo el simulador recibe una traza, es decir, un fichero con todos los accesos a memoria, y permite simular la evolución de la caché.

La simulación basada en traza proporciona una visión de los accesos a memoria simplificada. Supone que los accesos a memoria son independientes de la CPU y de la jerarquía de memoria empleada, lo cual no es cierto. Por ejemplo, dependiendo de la microarquitectura empleada y de la configuración de caché, pueden aparecer detenciones que modifican la secuencia de accesos a memoria. No obstante, la simulación basada en traza es útil por su simplicidad desde el punto de vista docente para entender el funcionamiento de la caché.

Para situar el simulador en el modo de simulación de caché basada en traza es necesario abrir un fichero con extensión `.cache` con la traza a simular. Para empezar, vamos a generar un archivo de traza a partir de la simulación del programa `3-1upper-case.asm`.

- Abre de nuevo el archivo `3-1upper-case.asm` y pulsa sobre el icono en la barra de menú. Aparece un diálogo en el que debes introducir el nombre del archivo de traza. Guarda la traza con el nombre `3-1mem-trace`, automáticamente se añadirá la extensión `.cache`.
- Abre ahora el archivo de traza generado pulsando sobre el icono en la barra de menú. Verás que no te aparece el archivo creado, pues el filtro que emplea el diálogo de selección de archivo se corresponde con el último tipo de archivo abierto. Cambia el filtro para mostrar los archivos con extensión `.cache` y abre el archivo de traza. La vista del simulador `pyMIPS64` cambia, aunque mantiene la filosofía del modo de simulación MIPS64.
- Pulsa sobre el icono 👁 y activa la casilla de `Locality of reference`. Aparece un nuevo panel en el que se muestran dos gráficas de accesos a memoria, una para el código y otra para los datos. Comprueba que las gráficas se corresponden con las que habías representado en la hoja de cálculo. Estas gráficas permiten cierta interacción para realizar *zoom* empleando los botones del ratón. Se puede recuperar el estado original pulsando sobre la `A` que aparece en la esquina inferior izquierda de cada gráfica. La figura 4 muestra la vista del simulador tras ajustar el tamaño de los paneles.

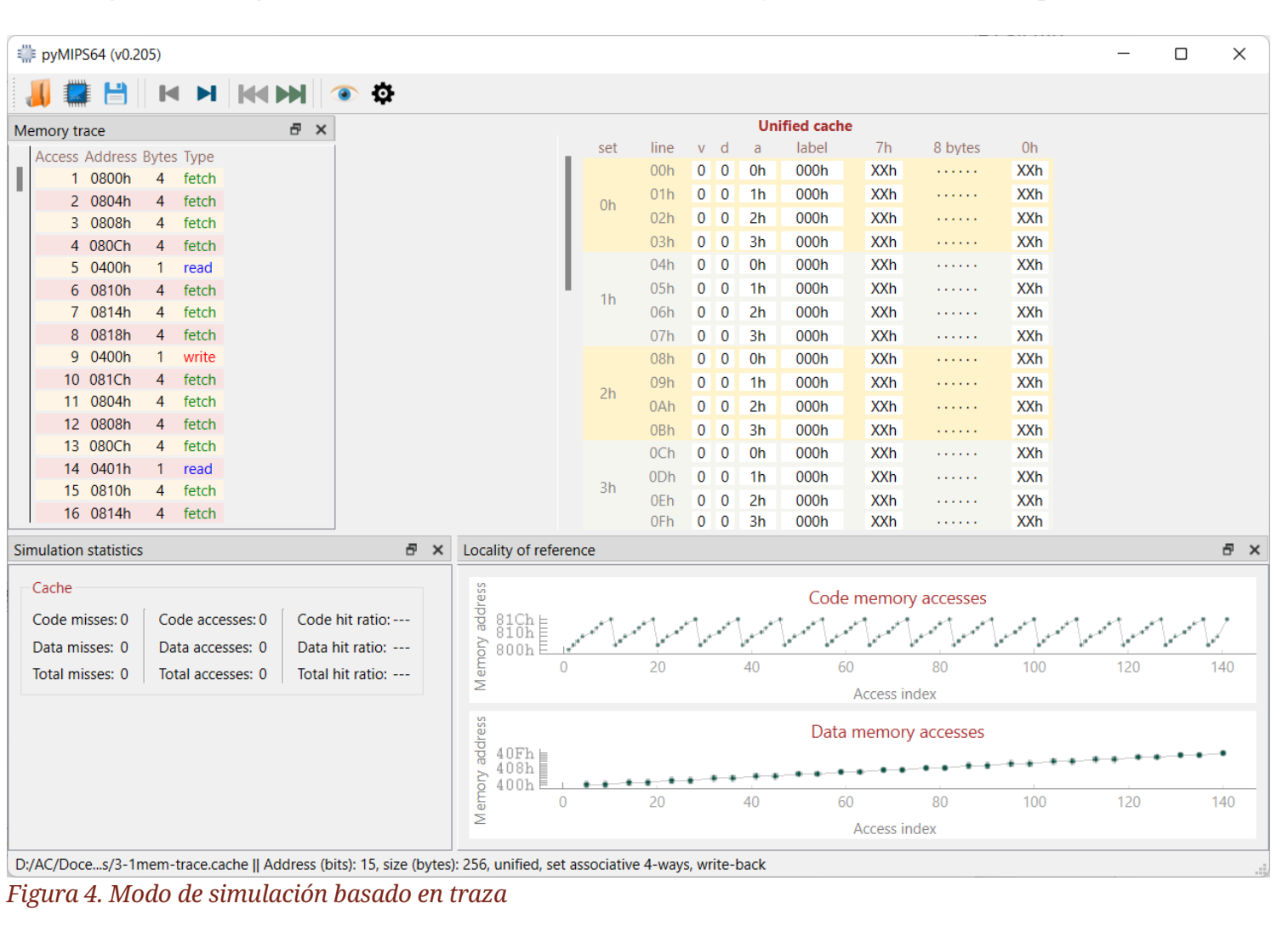

Figura 4. Modo de simulación basado en traza

El panel de la izquierda, denominado `Memory trace`, contiene la traza de memoria en un formato legible. Cada línea se corresponde con un acceso a memoria, definido por una dirección, el número de bytes accedidos y el tipo de acceso: *fetch* para acceso a código, *read* para lectura de datos y *write* para escritura de datos. Una restricción a tener en cuenta es que el simulador supone que cada dirección de memoria hace siempre referencia a 1 byte.

- El panel que se encuentra en la parte superior derecha es el panel central y en él se muestra el estado de la caché. Al no haber configurado la caché se emplea una configuración por defecto que puedes observar pulsando sobre el botón de la barra de menú. ¿Cuántos conjuntos tiene la caché? Responde en el cuestionario: pregunta 22. Cada conjunto de caché aparece numerado en la columna `set` en el panel central, además aparece con un sombreado que lo diferencia. Con la información proporcionada, comprueba tu respuesta.

Toda la información que almacena la caché aparece en el panel central en negro sobre fondo blanco. El resto se corresponde con información de apoyo que proporciona el simulador, como el número de conjunto, línea de caché, etc.

- Los campos `v`, `d`, `a` y `label` son metadatos de la caché. Los datos que almacena realmente, asociados a las direcciones de memoria cacheadas son los que aparecen con los valores `XXh`. Estos valores no son relevantes para el simulador, por lo que no tendrán nunca valores numéricos. El objetivo es resaltar la información que almacena cada línea de caché. ¿Cuántos bytes almacena cada línea de caché? Responde en el cuestionario: pregunta 23.

Vamos a comenzar con la simulación.

- Pulsa sobre el botón ▶|. El efecto es que se simula el primer acceso a memoria que ahora aparece resaltado con fondo azul en la traza y también se muestra como una línea en el panel de localidad. La visualización de la caché muestra en rojo los cambios que se han producido en la misma durante este acceso. ¿Qué crees que ha producido, acierto o fallo de caché? Responde en el cuestionario: pregunta 24.

Los fallos de caché se muestran de varias formas. En primer lugar, con el borde rojo en la línea de la traza simulada. También se puede ver en los bytes de datos de la línea de caché afectada, pues aparecen en fondo rojo indicando que ha variado su contenido, tal como ocurre en un fallo de caché. Finalmente, en el panel de estadísticas se muestra en rojo el número de fallos de caché si en el último acceso se ha producido un fallo.

Aunque el simulador nos indica directamente la evolución de la caché, desde el punto de vista educativo es más conveniente que el alumno prediga el estado de la caché para cada acceso a memoria antes de simularlo. En este sentido, el simulador serviría como comprobación.

- Antes de simular el segundo acceso a memoria, escribe sobre el papel la dirección y descomponla en los campos etiqueta, conjunto y desplazamiento a partir de la configuración de caché dada. Responde en el cuestionario: pregunta 25.
- Comprueba si la descomposición es correcta posicionando el puntero del ratón sobre el segundo acceso. Verás que aparece un mensaje emergente (*tooltip*) con la descomposición calculada por el simulador.
- Para el segundo acceso, ¿se produce acierto o fallo de caché? Debes responder sin emplear el simulador y después comprobarlo simulando el acceso a memoria.

Una de las posibilidades que incluye el simulador es ir hacia atrás en la simulación. Esto nos será útil para repetir un acceso que no hemos entendido, pero también para detectar fácilmente reemplazos o actualizaciones, aunque estos serán tratados en la próxima sesión práctica.

- Pulsa sobre el botón |◀ para volver hacia atrás y después sobre el botón ▶| para repetir el acceso.

Fíjate además cómo aquellos accesos que han sido simulados y producen fallo de caché permanecen con el borde rojo para facilitar su seguimiento. Además, durante la simulación en el panel de estadísticas se muestran estadísticas sobre los aciertos y fallos de caché.

- Predice sobre el papel la evolución de la caché en los siguientes 2 accesos. Debes predecir no sólo si hay acierto o fallo de caché, sino también cómo evoluciona el estado de la caché. Para cada acceso comprueba si la predicción fue correcta y si no lo fue, vuelve hacia atrás e inténtalo de nuevo.
- Simula la traza hasta el final pulsando sobre el botón ▶▶|. ¿Cuál es la tasa de aciertos de la traza con la configuración de caché dada? Responde en el cuestionario: pregunta 26.
- Para finalizar con la descripción de la funcionalidad del simulador, éste dispone de un botón |◀◀ para ir al comienzo de la simulación. Pruébalo.
- También dispone de un botón 💾 para guardar la traza actual y la configuración de memoria en un archivo `.cache`. Cambia la configuración de la caché, guarda la traza actual y la nueva configuración en el archivo `3-1trace-cache.cache`. Una vez guardado sal del simulador, vuelve a abrirlo y carga el fichero de caché guardado. Comprueba que la configuración contiene los cambios introducidos.

La simulación de la caché basada en la traza, aunque tiene sus limitaciones, resulta útil para estimar el tiempo de acceso a memoria durante la ejecución de un programa. Esta estimación puede realizarse empleando la expresión del tiempo medio de acceso a la jerarquía proporcionado en la parte teórica de la asignatura.

En la expresión vista en teoría para calcular el tiempo medio de acceso a memoria, aparece el parámetro B, que indica el número de bytes que constituyen la línea de caché. En la realidad, este valor B se considera como el número de accesos a memoria necesarios para transferir la línea caché. Este número de accesos depende por tanto: del ancho de línea y del bus de datos utilizado para transferir la información. Teniendo en cuenta esta consideración, el tiempo necesario para satisfacer un fallo de caché sería el producto del tiempo de acceso a memoria principal por el número de accesos.

En el ejemplo considerado en el simulador, se supone un bus de datos de 64 bits, un tiempo de acceso a la memoria principal de 8 ciclos, y la memoria caché se ha configurado con un ancho de línea de 8 bytes. Con estos datos, el tiempo necesario para satisfacer un fallo de caché (**Cache miss time** en el simulador) sería: **Main memory access time** x **Line size** / **Bus size** (en bytes). Esto es igual a 8 ciclos x 8 bytes / 8 bytes = 8 ciclos.

- ¿Cuál es el tiempo medio de acceso a la jerarquía de memoria en ciclos de reloj para la traza dada con la configuración de memoria empleada? Debes suponer que son necesarios 8 ciclos de reloj para satisfacer un fallo de caché, es decir, para mover un bloque de memoria principal a caché. Además, como dato adicional, no se producen actualizaciones de memoria principal. Responde en el cuestionario: pregunta 27.

En general, el valor obtenido es una estimación y no tiene por qué coincidir con el valor exacto, pues tal como se comentó el fichero de traza depende de la configuración de memoria empleada, por lo que al cambiar la configuración de memoria la traza puede cambiar. Además, si en lugar de emplear una arquitectura von Neumann para la memoria se emplea una arquitectura Harvard, se pueden solapar dos accesos a memoria, lo que reduce el tiempo medio de acceso.

Finalmente, para observar el efecto de la localidad sobre el funcionamiento de la jerarquía vamos a suponer una traza en la cual los accesos a memoria son aleatorios. La configuración de la memoria seguirá siendo la misma y recuerda que puedes visualizarla pulsando sobre el botón .

- ¿Cuál sería la tasa de aciertos de la caché si los accesos a memoria fuesen aleatorios? Responde en el cuestionario: pregunta 28. ¿Cuál sería el tiempo medio de acceso a la jerarquía de memoria en este caso? Como en en ejemplo anterior, debes considerar un tiempo de fallo de caché de 8 ciclos y que no hay actualizaciones. Responde en el cuestionario: pregunta 29 en ciclos de reloj.

La consecuencia que se extrae es que la localidad es clave para conseguir un funcionamiento adecuado de la jerarquía de memoria y conseguir que el tiempo medio de acceso a la memoria sea cercano al de acceso a la memoria caché.

Ejercicios

La figura 3 mostraba 10 escenarios posibles para la obtención del tiempo de acceso a la jerarquía de memoria formada por la memoria caché y la memoria principal.

- Abre el archivo `3-1upper-case-config.asm` y prueba con diferentes estrategias de escritura y de *allocation*. Antes de simular debes predecir el número de ciclos de reloj que requerirá un acceso a memoria de lectura o escritura, con acierto o fallo de caché, sin actualización y con actualización (si es posible). Una vez echa la predicción debes comprobarla con los datos de la figura 3 y finalmente llevando a cabo la simulación del archivo `3-1upper-case-config.asm`.

---

[1] Los códigos ASCII de las letras mayúsculas y minúsculas solo se diferencian en el bit 5, comenzando en 0. En el caso de las letras mayúsculas este bit es 0 y en el de las minúsculas es 1. Por tanto, se puede pasar de mayúscula a minúscula y viceversa modificando apropiadamente dicho bit.